Memory faults may not cause significant damages in small programs, but can be extremely dangerous in safety-critical applications and can have disastrous consequences; for instance, a segmentation fault in a medical application may lead to loss of lives. Hence, one must be extremely careful about memory leaks.

Detecting memory errors using Valgrind

In this section, let us explore how to use Valgrind to detect memory errors in a program written in C/C++. Apart from the MemCheck tool, the Valgrind distribution also includes thread error detectors, a cache and branch-prediction profiler, a call-graph generating cache and branch-prediction profiler, a heap profiler and three experimental tools: a heap/stack/global array overrun detector, a second heap profiler that examines how heap blocks are used, and a SimPoint basic block vector generator. Valgrind-3.8.1 is the latest stable version, which has been used for this article. The following platforms support Valgrind: X86/Linux, AMD64/Linux, ARM/Linux, PPC32/Linux, PPC64/Linux, S390X/Linux, MIPS/Linux, ARM/Android (2.3.x and later), X86/Android (4.0 and later), X86/Darwin and AMD64/Darwin.

Debugging with Valgrind using MemCheck

Unlike Java, languages like C or C++ do not have a garbage collector, which is an automatic memory manager for collecting memory occupied by unused objects in the program. Hence, there exists a significantly higher chance for memory faults to occur. One major issue with memory faults is that the error leads to a failure only during runtime. Thus, tools like Valgrind play a major role in detecting memory faults, without which debugging such errors becomes troublesome.

Given below is a demonstration of how to use Valgrind with the following code. It explains the scenarios listed earlier:

```
#include<iostream>
#include<stdlib.h>
using namespace std;

int main()
{
    int *x;
    x = new int(20); // no delete() used to release memory
allocated
    return 0;
}
```

Let us compile the above code using the following command:

```
$ g++ -g eg1.cpp -o eg1
```

To analyse this program using Valgrind, run the following command:

```
$ valgrind --tool=memcheck --leak-check=yes ./eg1
```

You will get the following output:

```
==5215== Memcheck, a memory error detector
==5215== Copyright (C) 2002-2012, and GNU GPL'd, by Julian Seward
et al.
==5215== Using Valgrind-3.8.1 and LibVEX; rerun with -h for
copyright info
==5215== Command: ./eg1
==5215==
==5215==
==5215== HEAP SUMMARY:
==5215==     in use at exit: 4 bytes in 1 blocks
==5215==   total heap usage: 1 allocs, 0 frees, 4 bytes allocated
==5215==
==5215== 4 bytes in 1 blocks are definitely lost in loss record 1
of 1
==5215==    at 0x402B87E: operator new(unsigned int) (vg_replace_
malloc.c:292)
==5215==    by 0x8048528: main (eg1.cpp:7)
==5215==
==5215== LEAK SUMMARY:
==5215==    definitely lost: 4 bytes in 1 blocks
==5215==    indirectly lost: 0 bytes in 0 blocks
==5215==      possibly lost: 0 bytes in 0 blocks
==5215==    still reachable: 0 bytes in 0 blocks
==5215==         suppressed: 0 bytes in 0 blocks
==5215==
==5215== For counts of detected and suppressed errors, rerun
with: -v
==5215== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0
from 0)
```

The number 5215 is the process ID of the program. Additionally, the tool provides information about various other properties of the program like heap summary, leak summary and error summary. Heap summary provides details regarding calls to *malloc/new/free/delete*. In the above output, the count of memory allocations and frees is mentioned; if not the same, it indicates a memory fault. The leak summary gives the amount of memory leaked; in the above example, this is 4 bytes, as shown. The error summary provides an overview about the total number of errors.

In this example, the memory allocated is not released using the *delete()* instruction. Thus, the 4 bytes are considered as 'definitely lost' as given in the leak summary, which indicates that your program is leaking memory.

Let us complicate our example code by adding the following, in order to consider Scenario 2: